Time-Varying Nonlinear Factor Models via Deep Autoencoders

M. Ladd

Preliminary & comments welcome

Brown bag seminar, October 20, 2026

Introduction

Consider household consumption.

- When credit is easy, an extra dollar of income might raise consumption by 80 cents. When credit is tight, maybe 30.
- The response also isn't linear *within* a regime: the same dollar does different things depending on whether the household is already liquidity-constrained.
- Now zoom out to thousands of goods across thousands of households. The consumption of each good responds to a small number of *shared drivers* — income, credit conditions, prices.

Exposure to a small number of common drivers can plausibly be **nonlinear**, *and can plausibly* **shift over time**.

Factor models are one way economists summarize this structure. But the standard model assumes the exposure — the *loading* — is **linear** in the drivers and **constant** over the sample. When both fail, the recovered common component is wrong, and downstream estimates inherit the error.

Research question

What do we gain in recovering common components by allowing factor loadings to be nonlinear and time-varying?

What is a common component?

Why nonlinearity? Why time variation?

Why nonlinearity?

- A factor model says an observation is $X = f(F)$, where F is a small set of shared drivers and f is the *loading*
- The standard assumption is f linear: $X = \lambda' F$
- But economic relationships are often nonlinear: convex payoffs, saturating responses, fat tails
- Shen and Xiu (2026): when linearity fails, a deep autoencoder recovers f at the optimal nonparametric rate

Why time variation?

- The loading f is usually assumed fixed over the whole sample
- But economic exposures change: crisis correlations spike, monetary regimes differ, financial innovation reshapes credit access
- Su and Wang (2017): significant time variation in a US macro panel; local PCA corrects it
- Bates et al. (2013): even small instabilities can bias standard PCA

These are not separate problems. If loadings are nonlinear in the factors *and* shift with the regime, addressing only one leaves the bias intact.

Contributions

The gap

	Linear loadings	**Nonlinear loadings**
Static	Stock–Watson (2002); Bai (2003)	Shen and Xiu (2026)
Time-varying	Su and Wang (2017)	**This paper**

This paper

- **Theory.** Establishes a non-asymptotic rate of convergence for the MSE of the recovered common component, extending both frameworks to the joint case
- **Method.** A global autoencoder with time as an input is tractable at realistic T; a local analogue is not
- **Evidence.** Tests where the correction matters empirically, on panels chosen to span linear and nonlinear loading structure

Preview of findings

- **Simulations.** On data with nonlinear, time-varying loadings, our method reduces the MSE of the recovered common component by 20–25% relative to the best static method. On linear, static data, no gain — as it should be.
- **Empirics.** On a panel whose loadings appear nonlinear, out-of-sample R^2 improves by 3.6 percentage points over static PCA. On panels whose loadings appear linear, R^2 is unchanged.
- **Caveat.** The gain does not carry over to minimum-variance portfolios. MVP Sharpe depends on the small eigenvalues of the precision matrix, which is not the quantity the theory targets.

Related literature

- **Linear factor models** (Bai and Ng 2002; Stock and Watson 2002; Bai 2003)
  Do: linear, static loadings. We do: nonlinear, time-varying loadings.
- **Time-varying factor models** (Su and Wang 2017; Bates et al. 2013; Breitung and Eickmeier 2011)
  Do: linear loadings that vary in t, estimated locally. We do: nonlinear loadings, estimated globally, at a faster rate.
- **Nonlinear factor models / autoencoders** (Shen and Xiu 2026; Gu, Kelly, and Xiu 2021)
  Do: nonlinear loadings, static. We do: extend to time-varying loadings.
- **Deep learning theory for nonparametric estimation** (Kohler and Langer 2021; Schmidt-Hieber 2020; Farrell, Liang, and Misra 2021)
  The approximation and concentration toolbox we use.

The model

A nonlinear, time-varying factor model:

$$X_{it} \;=\; \underbrace{X^{\star}_{it}}_{\text{common}} \;+\; U_{it}, \qquad X^{\star}_{it} \;=\; \varphi_i(F_t,\, t/T)$$

$i = 1, \ldots, N$ units; $t = 1, \ldots, T$ periods; $F_t \in \mathbb{R}^K$ latent factors; $s_t = t/T$.

Two departures from the standard linear static model:

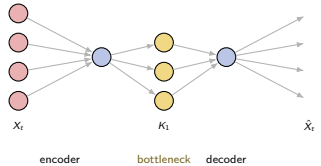
- **Nonlinearity.** $\varphi_i(\cdot, s)$ is an unknown function, not a linear map $\Lambda_i' F$.
- **Time variation.** $\varphi_i(\cdot, s)$ depends on $s = t/T$.

Nests the static linear model ($\Lambda_i' F$), the time-varying linear model ($\lambda_i(t/T)' F$) of Su and Wang (2017), and the static nonlinear model ($\varphi_i(F)$) of Shen and Xiu (2026) as special cases.

Why $s = t/T \in [0,1]$ and why smoothness. Su and Wang (2017), following Robinson (1989, 1991), rescale time so that locally around any fixed s the effective sample grows with T, which identifies the loading in the limit. The loading φ_i is assumed β-smooth in both (F, s) — the Hölder class targeted by neural-network approximation results (Stone 1982; Kohler and Langer 2021).

The estimand. The common component $X^{\star}$. As in every factor model, F_t and φ_i are not separately identifiable; $X^{\star}$ is.

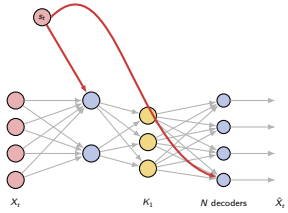
An autoencoder is a nonlinear factor model



- **Linear activations + squared loss ≡ PCA** (Baldi and Hornik 1989). Bottleneck width K_1 = number of factors.
- **Nonlinear activations:** same architecture, encoder learns a nonlinear map. The bottleneck plays the role of F_t; the decoder plays the role of φ_i.
- K **vs.** K_1**:** K is the true factor dimension, $K_1 \geq K$ is user-chosen. In nonlinear factor models, K is *not identified* (Shen and Xiu 2026, §3.2), so K_1 is treated as a hyperparameter. Over-specifying costs a linear $N^{-1}K_1$ penalty in the rate; it does not break the estimator.

Why an autoencoder and not a sieve?

Our architecture: disjoint output, time-conditioned



- **Disjoint output.** N separate decoder subnetworks, one per output, rather than one fully-connected decoder. Shen and Xiu (2026) show this retains the optimal nonparametric approximation rate while reducing parameter count by a factor of N. The reduced parameter count is what keeps the estimator trainable at finite T; the approximation guarantee is what lets the rate result inherit.
- **Time conditioning.** $s_t = t/T$ enters as an input to *both* the encoder and the decoder. The decoder captures time-varying loadings; the encoder captures time-varying factor composition — which units dominate the cross-section shifts with the regime, as it does in the crypto panel. A single model is trained on all T periods.

Why global, not local?

Assumptions and identification

Assumptions (four on this slide; full list in appendix)

- **Smoothness.** $\varphi_i \in \mathcal{H}^\beta([-B, B]^K \times [0, 1], B)$: smooth in both the factor F and the scaled time s.
- **Boundedness.** Factors on a compact set; sub-Gaussian noise with bounded spectral norm. Compact support is standard in nonparametric NN results and relaxable via truncation at negligible cost.
- **Weak dependence.** Mixing conditions used for the concentration step.
- **Optimization.** Training is non-convex; the rate holds up to an additive optimization slack, following Shen and Xiu (2026). Early stopping, ensembling, and validation manage this in practice.

Identification

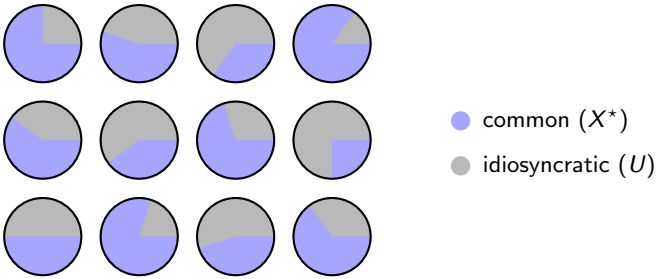
- Factors and loadings are never separately identified — standard, true for any factor model
- In our setting, the indeterminacy is a *time-indexed family* $F_t \mapsto \mu_t(F_t)$
- All results are stated for the common component $X^\star$, which is identified

Identification details Full assumptions

What is a common component?

Every observation splits into a part shared by many units (driven by K latent factors) and a part specific to the unit:

$$X_{it} = \underbrace{X^{\star}_{it}}_{\text{common}} + \underbrace{U_{it}}_{\text{idiosyncratic}}$$



- Each circle is one observed data point X_{it}. The blue area is the common component, the gray area the idiosyncratic. Units differ in how much of their variation is common.
- The **common component** is what a factor model tries to recover; the idiosyncratic part is what it treats as noise.
- Get $X^{\star}$ right, and downstream tasks — forecasts, pricing, hedging — inherit the right

Why an autoencoder and not a sieve?

The choice of model class.

- **Factors are unobserved.** A supervised sieve regresses X_{it} on $(F_t, t/T)$, but F_t is latent. The joint alternative — sieve both the loadings and the factors — is a non-convex problem with no natural dimension selection.
- **AEs handle this naturally.** An autoencoder learns both the latent factor (encoder) and the loading function (decoder) as a single optimization. Compositionality lets it adapt to effective low-dimensional structure without a prescribed basis.
- **Precedent.** Shen and Xiu (2026) establish the non-asymptotic rate for the AE under static nonlinear loadings; this paper extends their rate to the time-varying case.

Design choices within the autoencoder.

- **ReLU activations.** The Kohler and Langer (2021) approximation lemma is stated for ReLU networks; the rate result derives from that lemma.
- **Squared loss.** Targets the conditional mean of the common component, matching the theorem's estimand. Robust loss would target a different quantity.
- K_1 **as hyperparameter.** In nonlinear factor models, K is not identified (SX §3.2); K_1 is chosen by validation. The rate accommodates over-specification at a linear $N^{-1}K_1$ cost.

back

Why global, not local?

The direct analogue of Su and Wang (2017) would fit a separate autoencoder at each grid point in t/T, weighted by a kernel. At $T = 250$, the effective sample per grid point is roughly $Th \approx 30$ observations — far below what a network with thousands of parameters needs to train.

The global parametrization shares parameters across time: one encoder and one decoder set, evaluated at $s_t = t/T$. The effective sample is T, not Th.

This is what makes the estimator usable at realistic T. Without parameter sharing, the local analogue does not train.

back

Identification details

The indeterminacy. For any time-indexed family of invertible maps $\mu_t : \mathbb{R}^K \to \mathbb{R}^K$, the reparameterization

$$F_t \mapsto \mu_t(F_t), \qquad \varphi_i(\cdot, s_t) \mapsto \varphi_i(\mu_t^{-1}(\cdot), s_t)$$

leaves the common component $X_{it}^\star = \varphi_i(F_t, s_t)$ unchanged.

What is identified. The common component $X^\star$ is. The factor series $\{F_t\}$ and the loading functions $\{\varphi_i\}$ are identified only up to a time-indexed invertible transformation.

What this affects. Comparisons of $\hat{F}_t$ to F_t require a nonlinear alignment. Theoretically and empirically, we report results for the common component, which sidesteps the ambiguity.

How the indeterminacy shrinks in nested cases. In the linear static case, $\mu_t \equiv H$ (constant, matrix). In the linear time-varying case, $\mu_t = H_t$. In the nonlinear static case, μ is a nonlinear but time-invariant map. All three are special cases of the time-indexed family.

back

Full assumptions

- **Boundedness and noise.** Factors on $\mathcal{B} = [-B, B]^K$ a.s.; noise $U = \Sigma^{1/2} Z$ with sub-Gaussian Z; $\|\Sigma\|$ bounded.
- **Smoothness.** $\varphi_i \in \mathcal{H}^\beta(\mathcal{B} \times [0,1], B)$ for each i. β-smoothness in both the factor F and the scaled time s.
- **Encoder approximation.** There is a sparse linear map $W^\star$ and a smooth $\eta^\star$ such that $\eta^\star(W^\star \varphi(x, s), s) \approx x$ up to error ϵ_N. This quantifies the best possible recovery of the factors from the cross-section.
- **Weak dependence.** β-mixing with exponential decay; ρ-mixing summable. Used for the concentration step in the proof.

Notes on the theory-practice gap.

- *Sparsity.* The theory (SX Theorem 2) accommodates a sparsity parameter s in the encoder; in practice, weight decay acts as a smooth proxy, validated empirically.
- *Optimization.* The rate holds up to an additive optimization slack δ_{opt}; non-convexity is not accommodated by the theory but is managed empirically via early stopping, ensembling, and validation.

Network class and width scaling as in Shen and Xiu (2026), adapted to input dimension $K_1 + 1$ (the extra dimension is s). See the paper for the full statement.

back